

15. Avoid Deep Nesting



Bad:

bash

 Copiar

 Editar

```
GET /companies/1/departments/2/teams/3/members/4
```



Good:

bash

 Copiar

 Editar

```
GET /teams/3/members/4
```

Why? Deep nesting makes the URI hard to read and maintain.

14. Use Consistent Naming for IDs

 Bad:

```
bash
```

 Copiar  Editar

```
GET /users/id=1
```

 Good:

```
bash
```

 Copiar  Editar

```
GET /users/1
```

Why? Use simple path parameters for IDs.

13. Avoid Actions in the URI

 Bad:

bash

 Copiar  Editar

```
POST /users/1/activate
```

 Good:

bash

 Copiar  Editar

```
POST /users/1/actions/activate
```

or

bash

 Copiar  Editar

```
PATCH /users/1 { "active": true }
```

Why? REST APIs should focus on resources, not actions.

12. Keep URIs Stateless

 Bad:

pgsql

 Copiar  Editor

```
GET /users/1?session=abc123
```

 Good:

pgsql

 Copiar  Editor

```
GET /users/1 (Use authentication headers instead)
```

Why? Avoid stateful APIs; authentication should be handled using **headers**.

11. Use HTTP Status Codes Instead of Encoding Information in the URI

 Bad:

bash

 Copiar

 Editor

```
GET /users/1/status/404
```

 Good:

pgsql

 Copiar

 Editor

```
GET /users/1 → Returns `404 Not Found` if the user does not exist
```

Why? HTTP already provides status codes.

10. Use Singular Form for Singleton Resources



Bad:

```
bash
```

 Copiar

 Editar

```
GET /settings
```



Good:

```
sql
```

 Copiar

 Editar

```
GET /settings (if global settings)
```

```
GET /users/1/settings (if user-specific settings)
```

Why? Some resources only have **one instance per scope**.

9. Use Versioning in the URI or Headers

 Bad:

sql

 Copiar  Editar

```
GET /users (without versioning)
```

 Good:

bash

 Copiar  Editar

```
GET /v1/users
```

or

pgsql

 Copiar  Editar

```
GET /users (with `Accept: application/vnd.company.v1+json` header)
```

Why? It helps manage API changes without breaking clients.

8. Use Meaningful and Readable Resource Names

 Bad:

```
bash
```

 Copiar  Editar

```
GET /u/123/o/456
```

 Good:

```
bash
```

 Copiar  Editar

```
GET /users/123/orders/456
```

Why? Readable URLs improve API usability.

7. Use Consistent Lowercase Letters



Bad:

```
sql
```

 Copiar

 Editar

```
GET /Users/1/Orders
```



Good:

```
bash
```

 Copiar

 Editar

```
GET /users/1/orders
```

Why? URLs are case-sensitive, and lowercase improves consistency.

6. Avoid File Extensions



Bad:

pgsql

📄 Copiar ✎ Editar

GET /users.json



Good:

bash

📄 Copiar ✎ Editar

GET /users

Why? Content type should be handled via **headers** (`Accept: application/json`).

5. Use Hyphens (–) Instead of Underscores (_)



Bad:

```
bash
```

 Copiar

 Editar

```
GET /user_orders
```



Good:

```
sql
```

 Copiar

 Editar

```
GET /user-orders
```

Why? Hyphens improve readability in URLs.

4. Use Query Parameters for Filtering, Sorting, and Pagination

✓ Filtering:

pgsql

 Copiar  Editar

```
GET /users?role=admin
```

✓ Sorting:

sql

 Copiar  Editar

```
GET /users?sort=created_at&order=desc
```

✓ Pagination:

bash

 Copiar  Editar

```
GET /users?page=2&limit=10
```

Why? Query parameters should be used to modify the behavior of a resource retrieval.

3. Use Resource Hierarchies



Bad:

bash

 Copiar

 Editar

```
GET /orders?userId=1
```



Good:

bash

 Copiar

 Editar

```
GET /users/1/orders
```

Why? It makes relationships between resources clearer.

2. Use Plural Nouns

 Bad:

```
sql
```

 Copiar  Editar

```
GET /user/1
```

 Good:

```
bash
```

 Copiar  Editar

```
GET /users/1
```

Why? It keeps consistency across endpoints and avoids confusion.

1. Use Nouns, Not Verbs



Bad:

bash



Copiar



Editar

```
GET /getUsers
```

```
POST /createUser
```



Good:

bash



Copiar



Editar

```
GET /users
```

```
POST /users
```

Why? URIs should represent resources (nouns) rather than actions (verbs). The HTTP method already indicates the action.